EE6221 Robotics: Course Notes

Robotics Foundations and Manipulator Kinematics

Living course note

Updated: 24 August 2026

Scope. This review is grounded in the supplied course outline and the 156-page handout. It develops the manipulator material through direct and inverse kinematics, elementary trajectory planning, and differential motion/statics. Public exact-code files for robot control, mobile robots, and intelligent sensors are now archived under `resources/public-archive-2026-08-24/`, but they have not yet been reviewed or synthesised here.
Sources.

- `resources/course-outline-part-01.pdf`, p. 1: course sequence and references.
- `resources/lecture-01-manipulator-kinematics.pdf`, pp. 1–156: robotics overview, kinematics, trajectory planning, and Jacobian material. Page references below refer to this handout.

Contents

1 Course map and basic language

The supplied outline orders the course as: applications, manipulator kinematics, trajectory planning, robot control, and mobile robots. Kinematics is the bridge from actuator coordinates to a task written in physical space: it tells us where the tool is and how it is oriented for a given set of joint values.

An industrial manipulator is modelled as a serial chain of rigid *links* joined by actuated *joints*, with a base at one end and an end-effector (tool) at the other (pp. 1–2, 21–22). The two joint types used throughout are:

- **Revolute (R):** rotation about a joint axis; its variable is an angle.
- **Prismatic (P):** translation along a joint axis; its variable is a displacement.

The first three, "major", axes principally determine the reachable wrist positions; later wrist axes chiefly determine tool orientation (pp. 5–6). Common position architectures are PPP Cartesian, RPP cylindrical, RRP spherical/SCARA, and RRR articulated. A six-axis arm can generally place a tool at an arbitrary position and orientation within its workspace; more than six axes introduces kinematic redundancy (pp. 16, 98).

Workspace vocabulary. The *work envelope* is the locus of wrist positions reachable by the manipulator. *Reach* is a maximum distance, whereas *stroke* is an available travel distance; neither alone guarantees every orientation is achievable. Point-to-point motion only prescribes discrete destinations, while continuous-path motion prescribes the intervening path (pp. 12, 17–19).

2 Frames, rotations, and homogeneous transformations

2.1 Coordinates and the active/passive distinction

Let $F = \{\mathbf{f}_1, \mathbf{f}_2, \mathbf{f}_3\}$ and $M = \{\mathbf{m}_1, \mathbf{m}_2, \mathbf{m}_3\}$ be right-handed orthonormal frames. The superscript on ${}^F\mathbf{p}$ means "coordinates of the same geometric point $\mathbf{p}$ expressed in frame F." If M is rotated with respect to F, define

$$ {}^F\mathbf{p} = {}^F R_M \; {}^M\mathbf{p}. $$

Thus ${}^F R_M$ converts a coordinate description from the mobile frame to the fixed frame; it is not a second physical point. Its columns are the axes of M expressed in F:

$$ {}^F R_M = \begin{bmatrix} {}^F\mathbf{m}_1 & {}^F\mathbf{m}_2 & {}^F\mathbf{m}_3 \end{bmatrix}. $$

This convention matches the handout's rotation derivation (pp. 25–33).

For a proper rotation, $R^{\mathsf{T}} R = I$ and $R^{-1} = R^{\mathsf{T}}$. These are high-value checks: if either fails numerically by more than rounding error, the matrix is not a valid rigid-body orientation.

2.2 Fundamental and composite rotations

Using the right-hand rule, rotations about the x, y, and z axes are

$$ R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix}, \quad R_y(\phi) = \begin{bmatrix} \cos\phi & 0 & \sin\phi \\ 0 & 1 & 0 \\ -\sin\phi & 0 & \cos\phi \end{bmatrix}, \quad R_z(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi & 0 \\ \sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix}. $$

They map mobile coordinates into fixed coordinates under the convention above (pp. 28–33). Matrix multiplication is not commutative. Consequently, always state both the axis and the frame to which that axis belongs:

- rotation about a *fixed* frame axis: premultiply the current transformation;
- rotation about a *mobile* frame axis: postmultiply it.

This fixed/mobile rule is the main defence against incorrect rotation order (pp. 34–35). The handout uses a yaw-pitch-roll composition as an orientation convention (pp. 36–39); its angles must not be silently substituted for a different yaw-pitch-roll convention.

2.3 Homogeneous transformations

Rotation alone cannot translate an origin. Represent a point as $\bar{\mathbf{p}} = [\mathbf{p}^\top\ 1]^\top$ and a rigid transformation as

$$^{F}T_{M} = \begin{bmatrix} ^{F}R_{M} & ^{F}\mathbf{p}_{M} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \qquad ^{F}\bar{\mathbf{q}} = {}^{F}T_{M}\, {}^{M}\bar{\mathbf{q}}.$$

Here $^{F}\mathbf{p}_{M}$ is the origin of M written in F, not the position of an arbitrary point. Composition follows frame cancellation:

$$^{F}T_{N} = {}^{F}T_{M}\, {}^{M}T_{N}.$$

For $T = [R\ \mathbf{p}; \mathbf{0}^\top\ 1]$, the inverse is

$$T^{-1} = \begin{bmatrix} R^\top & -R^\top\mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix}.$$

These formulas and the fixed/mobile multiplication rule appear in pp. 39–52. A useful audit is dimensional and semantic: T_{AB} must map coordinates from B to A.

3 Direct kinematics and the D–H arm equation

3.1 The problem

Let $\mathbf{q} \in \mathbb{R}^n$ collect the joint variables. **Direct kinematics** asks for the tool pose relative to the base, given $\mathbf{q}$ (p. 23):

$$^{0}T_{n}(\mathbf{q}) = \begin{bmatrix} R(\mathbf{q}) & \mathbf{p}(\mathbf{q}) \\ \mathbf{0}^\top & 1 \end{bmatrix}.$$

The 3×3 block gives the tool-frame orientation and $\mathbf{p}$ gives the tool-tip position. This composite transform is the *arm matrix* (pp. 66–72).

3.2 Standard Denavit–Hartenberg parameters

The handout adopts the classical D–H convention. Attach a frame to each link so that z_{k-1} is the axis of joint k and x_k is the common normal between z_{k-1} and z_k (pp. 53–60). The adjacent-link relationship is described by:

parameter	meaning	joint-dependent when
θ_k	rotation about z_{k-1}	joint k is revolute
d_k	translation along z_{k-1}	joint k is prismatic
a_k	translation along x_k	never (link geometry)
α_k	rotation about x_k	never (link geometry)

Only one of θ_k and d_k is a joint variable. Starting at frame $k-1$, the transformation to frame k is

$$^{k-1}T_{k} = R_z(\theta_k)\,\mathrm{Tran}_z(d_k)\,\mathrm{Tran}_x(a_k)R_x(\alpha_k) = \begin{bmatrix} c_k & -s_k c_{\alpha k} & s_k s_{\alpha k} & a_k c_k \\ s_k & c_k c_{\alpha k} & -c_k s_{\alpha k} & a_k s_k \\ 0 & s_{\alpha k} & c_{\alpha k} & d_k \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

where $c_k = \cos\theta_k$, $s_k = \sin\theta_k$, $c_{\alpha k} = \cos\alpha_k$, and $s_{\alpha k} = \sin\alpha_k$ (pp. 66–70). Then

$$^{0}T_n(\mathbf{q}) = {}^{0}T_1\,{}^{1}T_2 \cdots {}^{n-1}T_n.$$

Practical workflow. Draw every z-axis first, then each common normal x-axis, and finally obtain y by the right-hand rule. Tabulate (θ, d, a, α) before multiplying matrices. A D–H frame assignment is not unique (pp. 61–65), but a consistent assignment yields the same physical tool pose.

3.3 Example: planar 2R position

For two coplanar revolute links of lengths a_1, a_2, with base angles q_1, q_2, the tool-tip position is

$$x = a_1 \cos q_1 + a_2 \cos(q_1 + q_2), \qquad y = a_1 \sin q_1 + a_2 \sin(q_1 + q_2).$$

This small example exposes two durable ideas: joint angles enter nonlinearly, and the second link's contribution is expressed with the accumulated angle $q_1 + q_2$. The handout applies the same product-of-transforms method to a five-axis articulated robot and a four-axis SCARA robot (pp. 74–86).

3.4 Tool orientation vectors

If $R = [\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{r}_3]$, its columns are respectively the tool *normal*, *sliding*, and *approach* vectors. The approach vector points along the tool roll axis; together the vectors form a right-handed orthonormal tool frame (pp. 57–58). For the handout's SCARA example, the approach direction is fixed vertically, which explains why it is appropriate for top-down light assembly but cannot realize arbitrary orientations (pp. 83–85).

4 Inverse kinematics

4.1 Statement, feasibility, and multiplicity

Inverse kinematics (IK) reverses the arm equation: given a desired position $\mathbf{p}_d$ and orientation R_d, find joint values $\mathbf{q}$ such that

$$^{0}T_n(\mathbf{q}) = \begin{bmatrix} R_d & \mathbf{p}_d \\ \mathbf{0}^\top & 1 \end{bmatrix}.$$

Manipulation tasks are naturally specified this way (pp. 87–89). Before solving, check:

1. the requested position lies inside the work envelope;
2. the requested orientation is realizable by the available orientation axes;
3. joint limits and collision constraints are respected (an engineering requirement beyond pure geometry).

The handout stresses the first two feasibility conditions and notes that arbitrary 3D pose requires at least six independent degrees of freedom (pp. 96–99).

IK can have no solution, one solution, or several solutions. The five-axis example has distinct elbow-up and elbow-down branches; a controller must select a branch compatible with limits and continuity from the current joint state (pp. 101–105). Use $\operatorname{atan2}(y, x)$ rather than $\arctan(y/x)$ where an angle's quadrant matters.

4.2 Analytic and numerical approaches

Analytic IK exploits a robot's particular geometry to isolate variables using trigonometry. It is fast and exact in ideal arithmetic, but is robot-specific. **Numerical IK** minimizes a pose error, for example

$$\min_{\mathbf{q}} \|\mathbf{w}(\mathbf{q}) - \widehat{\mathbf{w}}\|^2,$$

and is more broadly applicable but requires an initial guess and careful convergence handling (pp. 96–97).

For its articulated-robot examples, the handout represents a tool configuration by a six-vector

$$\mathbf{w} = \begin{bmatrix} \mathbf{p} \\ e^{q_n/\pi}\mathbf{r}_3 \end{bmatrix} \in \mathbb{R}^6,$$

where the scaled approach vector retains the approach direction and encodes the tool-roll angle q_n in its magnitude (pp. 91–95). Treat this as the course's convenient task-coordinate representation for these examples, not as a universal replacement for rotation matrices.

5 Trajectory planning

5.1 Frames for a pick-and-place task

A robust pick-and-place plan is more than a start and end point. The handout's minimal sequence uses a pick pose, lift-off pose, place pose, and set-down pose; via points are added to avoid obstacles (pp. 118–122). If $[R_{\text{pick}}, \mathbf{p}_{\text{pick}}]$ is the pick pose and $\mathbf{r}_3$ is its approach vector, a lift-off pose preserves orientation and moves back a safe approach distance v:

$$R_{\text{lift}} = R_{\text{pick}}, \qquad \mathbf{p}_{\text{lift}} = \mathbf{p}_{\text{pick}} - v\mathbf{r}_3.$$

The sign is meaningful: the approach vector points toward the object, so subtracting it retreats from the object under this convention.

5.2 Path versus trajectory

A *path* is spatial: a curve Γ in tool-configuration space. A *trajectory* adds time. The handout parameterizes a trajectory as $\mathbf{w}(t) = \Gamma(s(t))$, where $s : [0, T] \to [0, 1]$, $s(0) = 0$, and $s(T) = 1$ (pp. 123–125). The derivative $\dot{s}(t)$ is the speed profile. Keeping path and timing separate makes it easier to reuse a geometric path with a different timing law.

5.3 Cubic interpolation

For a segment between tool configurations $\mathbf{w}_0$ and $\mathbf{w}_1$, choose

$$\mathbf{w}(t) = \mathbf{a}t^3 + \mathbf{b}t^2 + \mathbf{c}t + \mathbf{d}, \qquad 0 \le t \le T,$$

and impose $\mathbf{w}(0) = \mathbf{w}_0$, $\dot{\mathbf{w}}(0) = \mathbf{v}_0$, $\mathbf{w}(T) = \mathbf{w}_1$, $\dot{\mathbf{w}}(T) = \mathbf{v}_1$. The coefficients are

$$\begin{aligned} \mathbf{d} &= \mathbf{w}_0, & \mathbf{c} &= \mathbf{v}_0, \\ \mathbf{a} &= \frac{2(\mathbf{w}_0 - \mathbf{w}_1) + T(\mathbf{v}_0 + \mathbf{v}_1)}{T^3}, & \mathbf{b} &= \frac{3(\mathbf{w}_1 - \mathbf{w}_0) - T(2\mathbf{v}_0 + \mathbf{v}_1)}{T^2}. \end{aligned}$$

For several knot points, adjacent cubics should be joined with continuous velocity and acceleration (pp. 130–132). Position continuity alone is usually not enough for smooth robot motion.

6 Differential motion, Jacobians, and singularities

6.1 Velocity kinematics

Let $\mathbf{x} = \mathbf{w}(\mathbf{q}) \in \mathbb{R}^6$ be the course's tool-configuration function. Differentiating gives

$$\dot{\mathbf{x}} = V(\mathbf{q})\dot{\mathbf{q}}, \qquad V(\mathbf{q}) = \frac{\partial \mathbf{w}}{\partial \mathbf{q}} \in \mathbb{R}^{6\times n}.$$

V is the tool-configuration Jacobian: column j is the instantaneous tool motion caused by unit rate at joint j (pp. 133–141). It is local: it maps velocities at the current configuration, not finite changes over an arbitrary interval.

When the rank and dimensions permit it, a Moore–Penrose pseudoinverse maps a desired tool rate to a joint rate,

$$\dot{\mathbf{q}} = V(\mathbf{q})^{\dagger}\dot{\mathbf{x}}.$$

For full column rank with $n \leq 6$, $V^{\dagger} = (V\mathsf{T}V)^{-1}V\mathsf{T}$; for full row rank with $n \geq 6$, $V^{\dagger} = V\mathsf{T}(VV\mathsf{T})^{-1}$. These are the resolved-motion rate-control relations developed in pp. 142–150. In implementation, an SVD-based pseudoinverse is safer near ill-conditioning than explicitly forming matrix inverses.

6.2 Singularities

A joint configuration is singular when the Jacobian loses its maximum possible rank:

$$\operatorname{rank} V(\widetilde{\mathbf{q}}) < \min(6, n).$$

At or near a singularity, some tool velocities cannot be produced and some required joint rates become very large. The handout distinguishes boundary singularities (on the workspace boundary) from interior singularities, where aligned axes can cause self-motion: joints move while the tool configuration does not (pp. 150–156).

For a square, nonsingular Jacobian, $|\det V|$ measures local volume scaling. More generally, use singular values to diagnose dexterity: a small smallest singular value means that the pose is close to singular. Trajectories should avoid these regions or slow down and use a damped numerical strategy.

7 Revision checklist

- Can I state the source and target frame of every R and T?
- Can I explain why fixed-axis operations premultiply while mobile-axis operations postmultiply?
- Can I construct one D–H row and identify the sole joint variable for R and P joints?
- Can I multiply D–H transforms into an arm matrix and read off $R(\mathbf{q})$ and $\mathbf{p}(\mathbf{q})$?
- Before IK, did I check reachability, orientation, joint limits, and solution branches?
- Can I distinguish a geometric path from a time-parameterized trajectory?
- Can I use $\dot{\mathbf{x}} = V\dot{\mathbf{q}}$ and explain what fails at a singularity?